

Evaluación Final: Proyecto Integrador Web con Datos e IA

- **Asignatura:** PRO301 · Taller de Aplicaciones para Internet
- **Unidad:** 03 · Datos, IA Aplicada y Proyecto Integrador
- **Fecha límite de entrega:** Martes 19 de mayo de 2026
- **Modalidad:** Práctica individual · Proyecto incremental
- **Docente:** Diego Obando

1. Sentido de la evaluación

La evaluación final consiste en cerrar y mejorar el proyecto que cada estudiante ha venido construyendo durante el módulo.

No se espera partir desde cero. La idea es demostrar evolución técnica: tomar la base desarrollada en las evaluaciones anteriores y convertirla en un sistema web más completo, más ordenado y más cercano a un producto real.

El proyecto debe integrar cuatro dimensiones:

- una interfaz web moderna y usable;
- una API o backend funcional;
- datos persistentes en una base de datos real;
- y una funcionalidad asistida por IA o automatización que aporte valor al usuario.

El foco no está en usar una herramienta específica por moda. El foco está en construir algo que funcione, que se entienda, que pueda explicarse técnicamente y que muestre criterio profesional básico.

2. Objetivo técnico

Desarrollar o consolidar un sistema web funcional que integre frontend, backend, persistencia de datos y una capacidad asistida por IA o automatización, cuidando la experiencia de usuario, la validación, la seguridad básica y la documentación técnica del proyecto.

Al finalizar, el proyecto debe permitir responder estas preguntas:

- ¿Qué problema resuelve la aplicación?
- ¿Qué datos guarda y por qué son importantes?
- ¿Qué operaciones permite realizar el usuario?
- ¿Qué parte del sistema usa IA o automatización?
- ¿Qué validaciones y defensas mínimas protegen el sistema?
- ¿Cómo se ejecuta y revisa el proyecto desde el repositorio?

3. Enfoque incremental

Esta evaluación final es incremental. Eso significa que se valora la evolución del trabajo ya realizado:

- si el proyecto comenzó como una landing o sitio estático, ahora debe comportarse como una aplicación;
- si ya tenía frontend y backend, ahora debe mejorar integración, orden, validación y experiencia;
- si ya tenía base de datos, ahora debe mostrar datos útiles y persistentes dentro de un flujo real;
- si ya usaba agentes o IA como apoyo de desarrollo, ahora debe documentar mejor qué se delegó, qué se revisó y qué se corrigió;
- si incorpora IA o automatización, esta debe estar conectada al propósito del producto y no aparecer como un botón aislado sin sentido.

Se puede cambiar o ajustar el tema del proyecto, pero no se recomienda empezar desde cero si eso reduce la calidad final. La prioridad es cerrar bien un producto demostrable.

4. Requisitos mínimos para que el proyecto sea evaluable

Para ser evaluado, el proyecto debe cumplir estos mínimos:

A. Repositorio ejecutable

- El código debe estar en GitHub.
- El repositorio debe ser público o estar compartido con el docente.
- Debe incluir un `README.md` con instrucciones claras para instalar, configurar y ejecutar el proyecto.
- Si el proyecto usa variables de entorno, debe incluir un archivo `.env.example`.
- No deben subirse claves reales, tokens, contraseñas ni secretos privados.

B. Frontend moderno

- Debe existir una interfaz web usable y responsiva.
- Se recomienda React, Next.js, Vite o una estructura basada en componentes.
- Si el estudiante mantiene una interfaz en HTML, CSS y JavaScript tradicional por continuidad del proyecto, debe justificarlo y demostrar orden, modularidad y buena experiencia de usuario.
- La interfaz debe mostrar estados importantes: carga, éxito, error o datos vacíos cuando corresponda.
- La aplicación debe verse como un producto real, no como una maqueta de prueba.

C. Backend, API o capa lógica

- Debe existir una capa que procese datos y conecte la interfaz con la persistencia.
- La comunicación debe usar JSON cuando corresponda.
- Las rutas, funciones o servicios deben tener nombres comprensibles.
- El backend debe validar entradas antes de guardar o procesar datos.
- Los errores deben responderse de forma controlada, no con fallos silenciosos ni errores genéricos sin manejo.

D. Datos persistentes

- El proyecto debe usar una base de datos real: PostgreSQL, MySQL, SQLite, MongoDB, Supabase, Neon u otra alternativa equivalente.
- Los datos deben sobrevivir al reinicio de la aplicación.
- La estructura de datos debe tener sentido para el negocio o problema elegido.
- Se valoran relaciones entre entidades cuando aporten al proyecto.

E. IA o automatización aplicada

El proyecto debe incorporar al menos una capacidad asistida por IA o automatización. Puede ser una de estas vías:

- integración con un LLM mediante API, por ejemplo OpenAI, Anthropic u otro proveedor;
- automatización de una tarea repetitiva o decisión del sistema;
- recomendación, clasificación, resumen, análisis, generación de texto o asistencia al usuario;
- uso de reglas inteligentes o lógica propia si resuelve una necesidad real del productoy;
- un modelo simple o prototipo entrenado, si está integrado a la aplicación y puede explicarse.

La funcionalidad debe tener entrada, procesamiento y salida visibles. No basta con agregar un botón que no afecte realmente al sistema.

Si se usa una API de IA:

- la clave debe permanecer en el backend o en variables de entorno;
- nunca debe quedar expuesta en el frontend;
- la respuesta debe manejarse con validación y control de errores;
- se debe explicar brevemente qué se envía al modelo y qué se recibe.

F. Seguridad y validación básica

El proyecto debe demostrar criterio mínimo de seguridad:

- no exponer secretos;
- validar datos de entrada;
- evitar confiar ciegamente en lo que viene desde el cliente;
- manejar errores sin romper toda la aplicación;
- evitar configuraciones peligrosas sin justificación, como CORS completamente abierto en un proyecto que se presenta como producto;
- cuidar datos sensibles si el proyecto maneja usuarios, correos, contraseñas o información personal.

5. Entregables

La entrega debe incluir:

1. Enlace al repositorio de GitHub.
2. `README.md` del proyecto con:
 - descripción del proyecto;
 - problema que resuelve;
 - stack utilizado;
 - instrucciones de instalación y ejecución;
 - variables de entorno necesarias;
 - explicación de la funcionalidad de IA o automatización;
 - breve explicación de cómo se usaron agentes o IA durante el desarrollo;
 - limitaciones conocidas o mejoras futuras.
3. Código fuente completo.
4. Base de datos configurada o instrucciones para crearla.
5. Evidencia mínima de funcionamiento: capturas, video corto, endpoints documentados, Swagger, archivo `.http`, colección de pruebas o instrucciones reproducibles.

6. Rúbrica de evaluación

Criterio	%	Excelente	Logrado	Insuficiente
Persistencia y modelo de datos	15%	Usa una base de datos real, con estructura coherente, tipos correctos, identificadores claros y relaciones cuando aportan al proyecto. Los datos persisten correctamente.	Usa base de datos real y guarda datos relevantes, aunque el modelo puede ser simple o tener detalles mejorables.	No usa base de datos real, los datos se pierden al reiniciar o la estructura no se entiende.
Backend, API e integración técnica	15%	La API o capa lógica está bien organizada, usa JSON, valida entradas, responde errores de forma controlada y conecta correctamente frontend y datos.	La integración funciona, pero con rutas, validaciones u organización todavía mejorables.	El frontend no se conecta realmente con el backend, hay datos hardcodeados o el servidor falla ante casos básicos.
Frontend moderno y experiencia de usuario	15%	La interfaz es clara, responsiva, ordenada y con apariencia de producto. Maneja estados de carga, error, éxito y datos vacíos.	La interfaz permite usar el sistema y es razonablemente clara, aunque puede mejorar en diseño, estados o responsividad.	La interfaz es confusa, no responsiva, incompleta o parece una maqueta sin flujo real de uso.
Capacidad de IA o automatización aplicada	20%	La funcionalidad de IA o automatización resuelve una tarea relevante del producto, está integrada al flujo principal, tiene entrada y salida claras, y se maneja con validación y errores.	La funcionalidad existe y se puede probar, pero su aporte al producto todavía es limitado o poco profundo.	La funcionalidad no existe, no funciona, es decorativa o no se conecta con el problema del proyecto.
Seguridad, validación y robustez	15%	Protege secretos, usa variables de entorno, valida datos en servidor, maneja errores, cuida CORS/permisos y evita exponer información innecesaria.	Tiene algunas validaciones y cuidado básico, pero mantiene riesgos o configuraciones débiles.	Expone claves, no valida datos, falla con entradas simples o deja errores críticos sin manejar.
Evolución incremental y criterio de producto	10%	Se nota una mejora clara respecto a entregas anteriores: mejor flujo, mejor arquitectura, mejor experiencia y decisiones justificadas.	Hay avance respecto al proyecto previo, aunque todavía se siente como una suma de partes.	No hay evolución clara, el proyecto parece improvisado o se reinició sin alcanzar una versión sólida.
Uso documentado de IA/agentes con criterio	5%	Explica qué se usó con IA/agentes, qué se revisó manualmente, qué se corrigió y qué decisiones tomó el estudiante.	Menciona uso de IA/agentes, pero con poca profundidad sobre validación o criterio propio.	No documenta el uso de IA/agentes o entrega código que no puede explicar.
Orden, Git y documentación de entrega	5%	Repositorio ordenado, commits razonables, README claro, <code>.env.example</code> , instrucciones reproducibles y sin archivos basura.	El repositorio permite revisar el proyecto, aunque la documentación u orden pueden mejorar.	Entrega desordenada, sin instrucciones claras, sin historial útil o difícil de ejecutar.

7. Aspectos que pueden subir la calidad de la entrega

Estos elementos no reemplazan los requisitos mínimos, pero pueden fortalecer la evaluación:

- usar relaciones entre tablas o colecciones cuando el dominio lo necesite;
- separar rutas, controladores, servicios o componentes;
- usar TypeScript, Zod, Pydantic, esquemas o validadores equivalentes;
- incluir Swagger, archivos `.http`, pruebas manuales documentadas o una colección de endpoints;
- manejar autenticación o roles si el proyecto lo justifica;
- usar respuestas estructuradas de IA en JSON cuando sea necesario;
- agregar límites o controles para evitar mal uso de la funcionalidad de IA;
- registrar errores o eventos importantes del sistema;
- mejorar la accesibilidad de formularios, botones, mensajes y navegación;
- documentar decisiones técnicas importantes en el README.

8. Aspectos que bajan la evaluación

Estos problemas afectan directamente la nota:

- entregar un proyecto que no ejecuta;
- entregar solo frontend sin backend ni persistencia;
- simular datos con arreglos locales cuando el requisito pide base de datos real;
- dejar claves de API, tokens o contraseñas subidas al repositorio;
- integrar IA solo como adorno, sin relación con el producto;
- depender completamente de código generado por IA sin entenderlo ni poder explicarlo;
- dejar textos de prueba visibles como `lorem ipsum`, `test`, `ejemplo`, `pendiente` o instrucciones internas;
- tener errores de consola o backend que impidan usar el flujo principal;
- no incluir README o instrucciones de ejecución;
- entregar fuera del plazo indicado.

9. Política de uso de IA y agentes

Está permitido usar herramientas como ChatGPT, Codex, Claude, Gemini u otros agentes para apoyar el desarrollo.

Se permite usarlas para:

- planificar arquitectura;
- explicar errores;
- generar una primera versión de código;
- refactorizar;
- escribir documentación;
- construir consultas;
- revisar seguridad;
- mejorar diseño o experiencia de usuario.

Pero el estudiante sigue siendo responsable del resultado final. Por lo tanto:

- debe entender el código entregado;
- debe ejecutar y probar el proyecto;
- debe corregir errores generados por la IA;
- debe revisar seguridad, variables de entorno y exposición de datos;
- debe poder explicar las decisiones técnicas principales.

En el README del proyecto debe existir una sección breve llamada `Uso de IA o agentes`, indicando:

- qué herramienta se usó;
- para qué se usó;
- qué parte fue revisada o modificada por el estudiante;
- qué límites o errores se detectaron.

10. Método de entrega

1. Subir el proyecto a GitHub.
2. Verificar que el repositorio sea público o esté compartido con el docente.
3. Confirmar que el `README.md` permite ejecutar el proyecto.
4. Enviar el enlace por el canal oficial de la asignatura.
5. La fecha límite es el **martes 19 de mayo de 2026**.

La entrega debe representar una versión final revisable del proyecto. No se evaluará solo la intención ni una explicación oral de algo que no quedó funcionando en el repositorio.